

# Multi-Layer Perceptrons for Function Approximation: Theory and Applications

Krishna Kumar

University of Texas at Austin

*krishnak@utexas.edu*

# Overview

- 1 Introduction and Motivation
- 2 Neural Network Fundamentals
- 3 Universal Approximation Theorem
- 4 Training Neural Networks
- 5 The Need for Depth

# Outline

- 1 Introduction and Motivation
- 2 Neural Network Fundamentals
- 3 Universal Approximation Theorem
- 4 Training Neural Networks
- 5 The Need for Depth

# The Challenge: 1D Consolidation Settlement

**Problem:** Predict settlement from sparse field measurements.

Terzaghi's one-dimensional theory gives the degree of consolidation as a series, with  $M = (2m + 1)\pi/2$  and  $T_v = c_v t / H_{dr}^2$ :

$$U(T_v) = 1 - \sum_{m=0}^{\infty} \frac{2}{M^2} e^{-M^2 T_v}$$

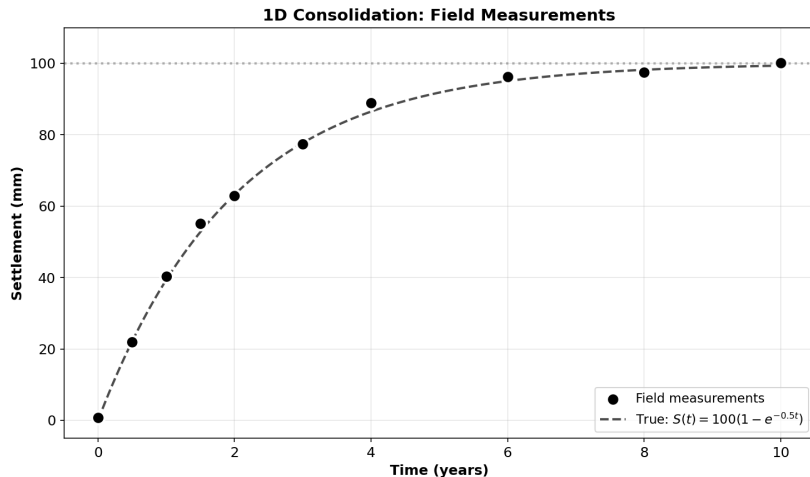
We generate data from a single exponential, which approximates the series:

$$S(t) = S_{\text{final}}(1 - e^{-\alpha t})$$

- $S_{\text{final}} = 100$  mm (ultimate settlement)
- $\alpha = 0.5 \text{ yr}^{-1}$  (rate constant,  $\alpha = \pi^2 c_v / 4 H_{dr}^2$  for the first mode)
- Sparse data: 10 measurements over 10 years
- The first mode alone reads  $U \approx 1 - (8/\pi^2) e^{-\pi^2 T_v / 4}$ . The single exponential drops that  $8/\pi^2$  prefactor.

**Goal:** Neural network learns continuous  $S(t)$  from discrete noisy measurements.

# Consolidation: Field Measurements



Sparse field measurements with noise

**Challenge:** Nonlinear, bounded physical problem with limited data.

# Outline

- 1 Introduction and Motivation
- 2 Neural Network Fundamentals**
- 3 Universal Approximation Theorem
- 4 Training Neural Networks
- 5 The Need for Depth

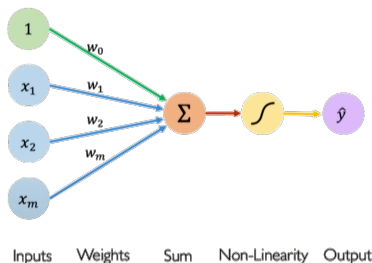
# The Perceptron: Basic Building Block

A perceptron computes:

$$z = \mathbf{w}^T \mathbf{x} + b = \sum_{i=1}^n w_i x_i + b$$

$$\hat{y} = g(z)$$

where  $g$  is the activation function.



Perceptron architecture

The diagram shows the mathematical equation for a perceptron's output, with arrows pointing to specific parts of the equation:  $\hat{y} = g \left( w_0 + \sum_{i=1}^m x_i w_i \right)$ . A purple arrow points to  $\hat{y}$  with the label 'Output'. A red arrow points to the summation term  $\sum_{i=1}^m x_i w_i$  with the label 'Linear combination of inputs'. A green arrow points to  $w_0$  with the label 'Bias'. A yellow arrow points to the function  $g$  with the label 'Non-linear activation function'.

**Key components:** Input vector, weights, bias, activation function, output.

# The Critical Role of Nonlinearity

**Without activation functions:** Multiple linear layers collapse to single linear transformation.

Consider two linear layers:

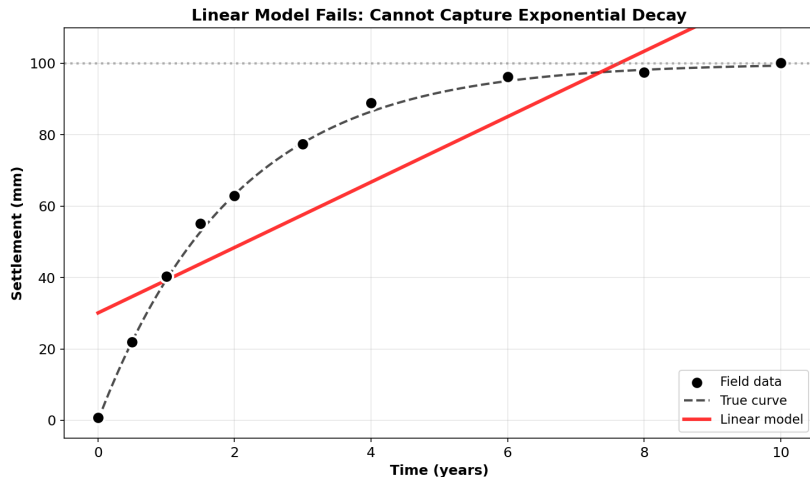
$$h_1 = W_1x + b_1$$

$$\begin{aligned} h_2 &= W_2h_1 + b_2 = W_2(W_1x + b_1) + b_2 \\ &= (W_2W_1)x + (W_2b_1 + b_2) \end{aligned}$$

This is equivalent to:  $h_2 = W_{eq}x + b_{eq}$  (still linear!)

**Problem:** Linear networks can only learn  $y = mx + b$ , but  $S(t) = 100(1 - e^{-0.5t})$  is curved.

# Linear Model Fails on Consolidation



Linear model  $\hat{S} = wt + b$  cannot capture exponential decay

**Conclusion:** Nonlinearity is essential for complex function approximation.

# Common Activation Functions

**Sigmoid:**  $\sigma(x) = \frac{1}{1+e^{-x}}$  (squashes to (0,1))

**Tanh:**  $\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$  (squashes to (-1,1))

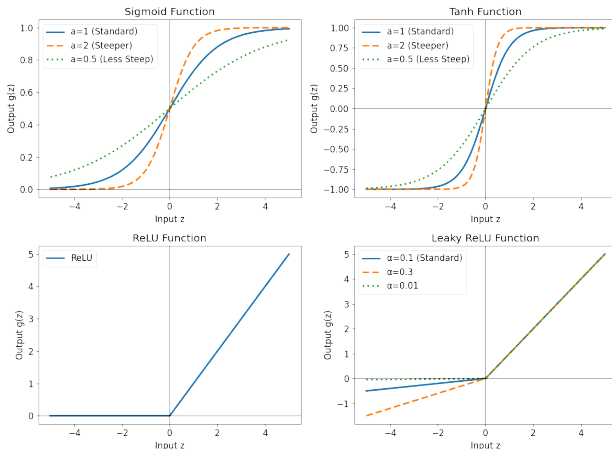
**ReLU:**  $f(x) = \max(0, x)$ , cheap to compute. Gradient 1 for  $x > 0$ , so no saturation on the positive side. Gradient 0 for  $x < 0$ . Not differentiable at 0, where frameworks return 0 by convention.

**Leaky ReLU:**  $f(x) = \max(\alpha_L x, x)$  with  $0 < \alpha_L \ll 1$ , typically 0.01 (prevents dead neurons). The leak  $\alpha_L$  is not the consolidation rate constant  $\alpha$ .

► ReLU Demo

# Common Activation Functions

Common Activation Functions (Parameterized)



Common activation functions

# Outline

- 1 Introduction and Motivation
- 2 Neural Network Fundamentals
- 3 Universal Approximation Theorem**
- 4 Training Neural Networks
- 5 The Need for Depth

# Universal Approximation Theorem

**Theorem (Cybenko, 1989):** A single hidden layer network with a sigmoidal activation approximates any continuous function on a compact domain to any accuracy. Leshno et al. (1993) extended this to every continuous non-polynomial activation.

$$F(x) = \sum_{i=1}^N w_i \sigma(v_i x + b_i) + w_0$$

**Mathematical statement:** For any continuous  $f : [0, 1] \rightarrow \mathbb{R}$  and  $\epsilon > 0$ , there exists  $N$  and parameters such that  $|F(x) - f(x)| < \epsilon$  for all  $x \in [0, 1]$ .

**The theorem says a network exists.** It gives no bound on  $N$ , no guarantee that training finds the parameters, and no control outside the domain (see 03d).

**Question:** How many neurons does the consolidation curve need? We measure that next.

# Single Hidden Layer Architecture

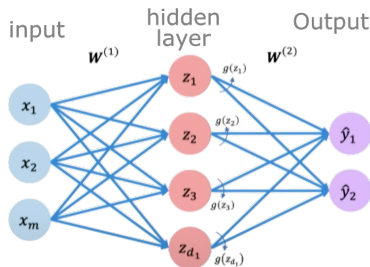
For 1D input  $x$ , a single-layer network with  $N_h$  hidden neurons:

$$\mathbf{z}^{(1)} = W^{(1)}x + \mathbf{b}^{(1)} \quad (\text{pre-activation})$$

$$\mathbf{h} = g(\mathbf{z}^{(1)}) \quad (\text{hidden layer output})$$

$$\mathbf{z}^{(2)} = W^{(2)}\mathbf{h} + b^{(2)} \quad (\text{output layer})$$

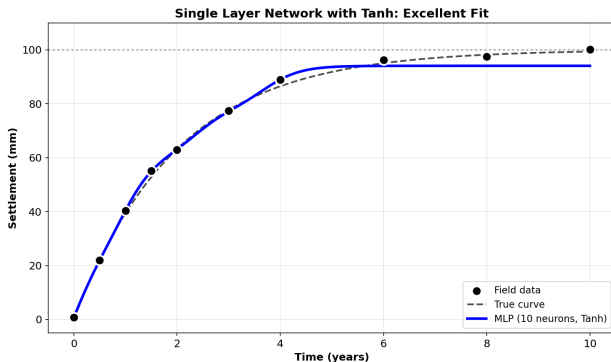
$$\hat{y} = z^{(2)} \quad (\text{final prediction})$$



Single hidden layer neural network

# Single Layer Network on Consolidation

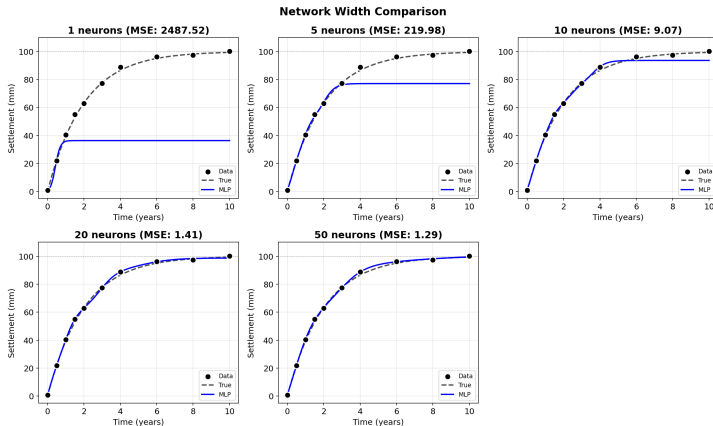
**Architecture:** Input  $\rightarrow$  Hidden (10 neurons, Tanh)  $\rightarrow$  Output



Tanh activation fits consolidation curve smoothly

**Result:** 10 neurons give a smooth fit that flattens at 94 mm, 6 mm below the 100 mm asymptote.

# Network Width Comparison

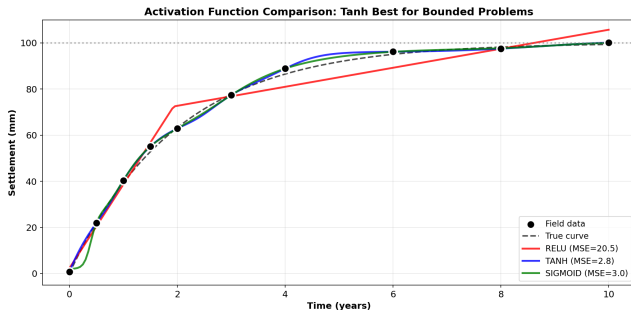


Consolidation approximation: 1, 5, 10, 20, 50 neurons

**Finding:** MSE is 9.07 at 10 neurons, 1.41 at 20, and 1.29 at 50. Returns flatten past 20 neurons.

# Activation Function Choice Matters

**Test:** ReLU vs Tanh vs Sigmoid on consolidation



Test MSE over 0 to 10 years: tanh 2.8, sigmoid 3.0, ReLU 20.5

**Rule:** The output layer is linear. The learned output weights set the asymptote, and the range of the activation does not. Tanh and sigmoid stay bounded outside the data. ReLU grows without limit and reaches 519 mm at 20 years in 03d.

# Outline

- 1 Introduction and Motivation
- 2 Neural Network Fundamentals
- 3 Universal Approximation Theorem
- 4 Training Neural Networks**
- 5 The Need for Depth

# Training Process

**Goal:** Find optimal parameters  $\theta^*$  that minimize loss function.

Loss function (MSE):

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N \left( s_i - \hat{S}(t_i; \theta) \right)^2$$

Optimization problem:

$$\theta^* = \arg \min_{\theta} \mathcal{L}(\theta)$$

## Training Steps:

- 1 Forward pass: compute predictions
- 2 Calculate loss
- 3 Backward pass: compute gradients
- 4 Update parameters
- 5 Repeat until convergence

# The Gradient Problem

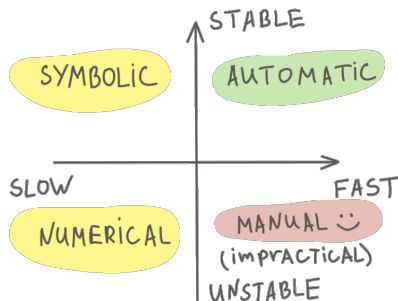
Training neural networks requires gradients:  $\frac{\partial \mathcal{L}}{\partial \theta}$  for all parameters  $\theta$ .  
Traditional approaches have fundamental flaws:

- **Manual:** Exact, but error-prone and doesn't scale
- **Symbolic:** Exact, but "expression swell"
- **Numerical:**  
$$f'(x) \approx \frac{f(x+h) - f(x-h)}{2h}$$
  - Inaccurate (rounding errors)
  - Expensive (multiple evaluations)

For a neural network with thousands of parameters:

$$\theta = \{W^{(1)}, \mathbf{b}^{(1)}, W^{(2)}, \mathbf{b}^{(2)}, \dots\}$$

## DIFFERENTIATION



We need a fourth approach: **Automatic Differentiation!**

# Automatic Differentiation: The Solution

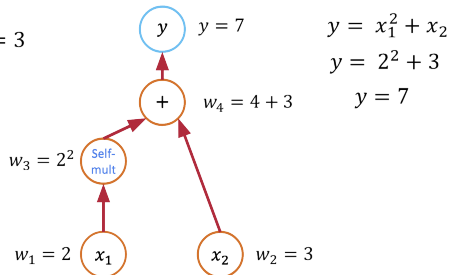
Every function is a computational graph of elementary operations.

Consider:  $y = x_1^2 + x_2$

**Evaluation Trace:**

- 1  $v_1 = x_1^2$
- 2  $y = v_1 + x_2$

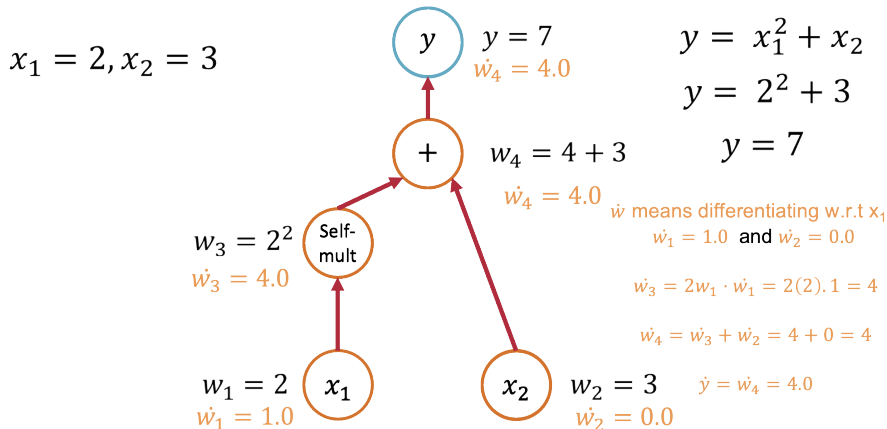
$$x_1 = 2, x_2 = 3$$



This decomposition is the key that makes AD possible. By applying the chain rule to each elementary step, we can compute exact derivatives.

# Forward Mode Automatic Differentiation

Forward mode AD propagates derivative information **forward** through the graph, alongside the function evaluation.



# Forward Mode: Step-by-Step Example

Let's compute  $\frac{\partial y}{\partial x_1}$  for  $y = x_1^2 + x_2$ .

1. Seed the input w.r.t.  $x_1$

Set  $\dot{x}_1 = 1$  and  $\dot{x}_2 = 0$ .

2. Forward Propagation

- $v_1 = x_1^2 \implies \dot{v}_1 = 2x_1 \cdot \dot{x}_1 = 2x_1 \cdot 1 = 2x_1$
- $y = v_1 + x_2 \implies \dot{y} = \dot{v}_1 + \dot{x}_2 = 2x_1 + 0 = 2x_1$

Result

The final propagated tangent  $\dot{y}$  is the derivative:  $\frac{\partial y}{\partial x_1} = 2x_1$ .

**Key Idea:** To get the derivative w.r.t.  $x_2$ , we would need a *new pass* with seeds  $\dot{x}_1 = 0, \dot{x}_2 = 1$ .

# Reverse Mode Automatic Differentiation

Reverse mode AD (or **backpropagation**) computes derivatives by propagating information **backward** from the output.

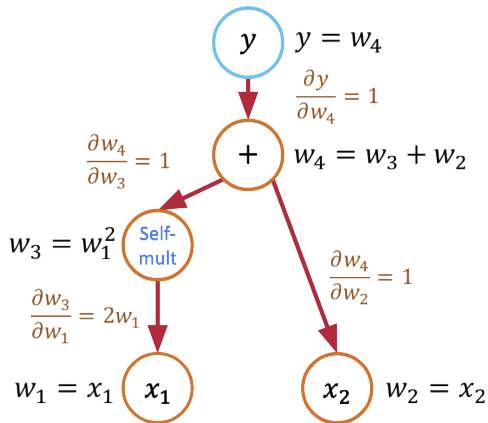
## Algorithm:

- 1 **Forward Pass:** Evaluate the function and store all intermediate values.
- 2 **Backward Pass:** Starting from the output, use the chain rule to propagate "adjoints" ( $\bar{v} = \frac{\partial y}{\partial v}$ ) backward through the graph.

# Reverse Mode: One Pass for All Gradients

The backward pass systematically applies the chain rule.

Let's trace the computation for  $y = x_1^2 + x_2$ :



$$y = x_1^2 + x_2$$

$$\nabla y = \left( \frac{\partial y}{\partial x_1}, \frac{\partial y}{\partial x_2} \right)$$

$$\nabla y = (2x_1, 1)$$

We can find the gradient with respect to the output  $y$

Using  $x_1 = 2, x_2 = 3$ :

$$\nabla y = (4, 1)$$

Reverse mode computes **all** partial derivatives in a single backward pass.



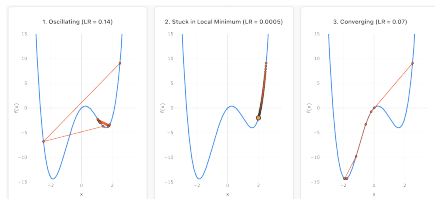
# Gradient Descent Algorithm

## Basic Algorithm:

- 1 Initialize weights randomly
- 2 Loop until convergence:
- 3     Compute gradient  $\frac{\partial \mathcal{L}}{\partial \theta}$
- 4     Update weights:  
$$\theta \leftarrow \theta - \eta \frac{\partial \mathcal{L}}{\partial \theta}$$
- 5     Return weights

$$\theta_{new} = \theta_{old} - \eta \nabla \mathcal{L}(\theta)$$

where  $\eta$  is the learning rate.



Gradient descent optimization

► SGD Demo

**Regression (Consolidation):** Mean Squared Error

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N} \sum_{i=1}^N \left( s_i - \hat{s}(t_i; \theta) \right)^2$$

**Binary Classification (Liquefaction):** Binary Cross-Entropy

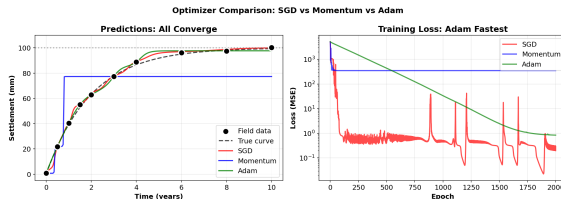
$$\mathcal{L}_{\text{BCE}} = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

**Why BCE for classification?**

- Probabilistic interpretation
- Stronger gradients for wrong confident predictions
- Works with sigmoid output  $\hat{y} \in (0, 1)$

# Optimizer Comparison

**Test on consolidation:** SGD vs Momentum vs Adam, all at  $\text{lr} = 0.01$



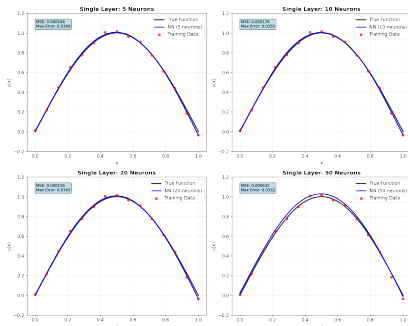
Consolidation fit and training loss. The panel titles in the figure are wrong: momentum does not converge, and its prediction is flat at 77 mm past 3 years.

**Result:** 03c gives final losses of 0.47 for plain gradient descent, 0.57 for Adam and 1067 for momentum. Plain gradient descent wins on final loss, Adam gets close in fewer epochs, and momentum diverges early.

**Recommendation:** A shared learning rate is not a fair comparison. Each optimizer needs its own rate. Start from Adam at  $\text{lr } 10^{-3}$ , and use  $10^{-2}$  for small smooth 1D fits.

# Width vs Approximation Quality

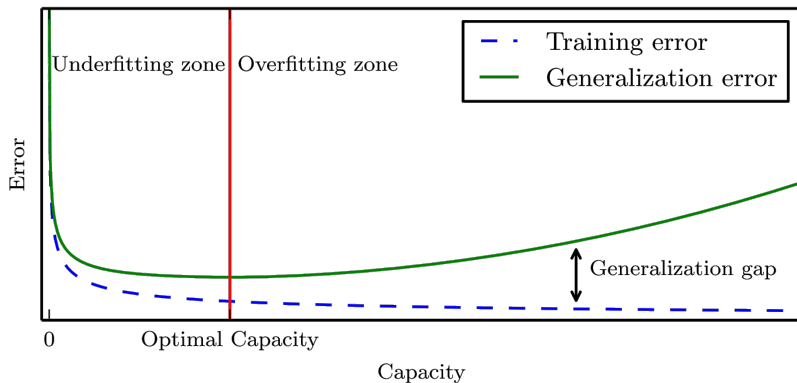
**Hypothesis:** More neurons  $\rightarrow$  better approximation. Single hidden layer, 5 to 50 neurons, target  $u(x) = \sin(\pi x)$  on  $[0, 1]$ . The consolidation slides use a different target.



MSE 0.000166, 0.000176, 0.000156, 0.000632 at 5, 10, 20, 50 neurons

**Result:** Fifty neurons is four times worse than twenty. The hypothesis fails here.

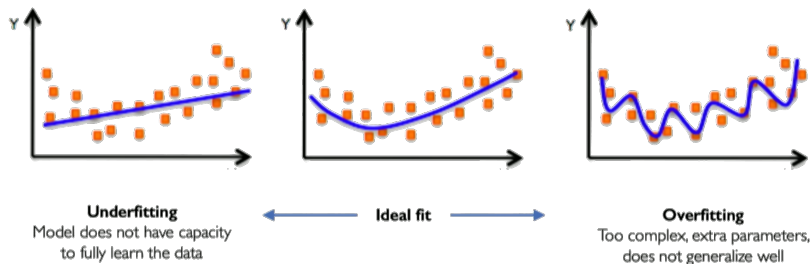
# Training/Validation



Training and validation convergence

# Overfitting and Model Capacity

**Problem:** High-capacity networks can memorize training data.

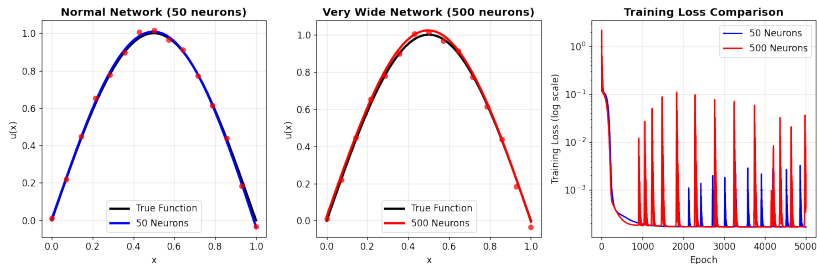


Under and overfitting illustration

**Detection:** Monitor validation loss during training.

**Solutions:** More data, regularization, early stopping, simpler architectures.

# Demonstrating Overfitting



Normal (50 neurons) vs very wide (500 neurons) network

**Observation:** Very wide networks may generalize worse despite lower training loss.

# Outline

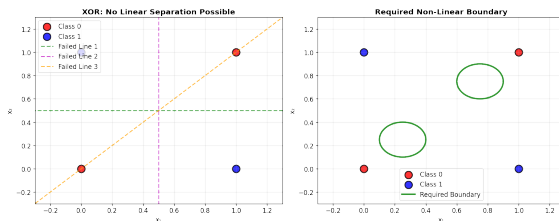
- 1 Introduction and Motivation
- 2 Neural Network Fundamentals
- 3 Universal Approximation Theorem
- 4 Training Neural Networks
- 5 The Need for Depth

# The XOR Problem: Historical Crisis

## XOR Truth Table:

| $x_1$ | $x_2$ | $y$ |
|-------|-------|-----|
| 0     | 0     | 0   |
| 0     | 1     | 1   |
| 1     | 0     | 1   |
| 1     | 1     | 0   |

**The Crisis:** No single line can separate these classes!



XOR is not linearly separable

# True Single-Layer vs Multi-Layer

## Critical Distinction:

- **True Single-Layer:** Input  $\rightarrow$  Output (NO hidden layers)
- **Multi-Layer:** Input  $\rightarrow$  Hidden  $\rightarrow$  Output (1+ hidden layers)

## True Single-Layer (fails):

$$y = \sigma(w_1x_1 + w_2x_2 + b)$$

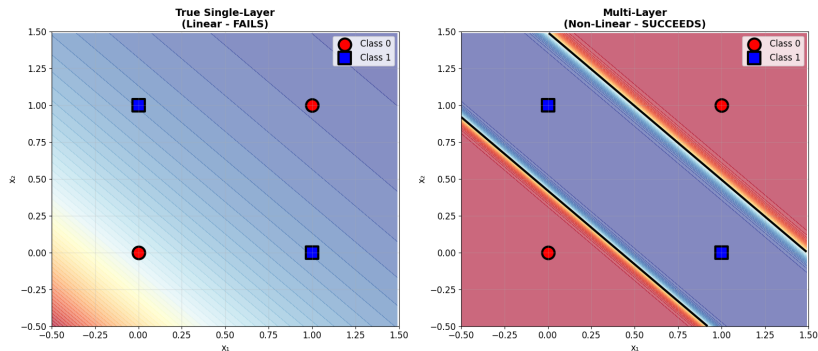
## Multi-Layer (succeeds):

$$h_1 = \sigma(w_{11}x_1 + w_{12}x_2 + b_1)$$

$$h_2 = \sigma(w_{21}x_1 + w_{22}x_2 + b_2)$$

$$y = \sigma(v_1h_1 + v_2h_2 + b_3)$$

# XOR Solution: Decision Boundaries



Linear (fails) vs Non-linear (succeeds) decision boundaries

**Key Insight:** Hidden layers enable curved boundaries through non-linear transformations.

## XOR Decomposition:

$$h_1 = \sigma(20x_1 + 20x_2 - 10) \quad (\text{OR})$$

$$h_2 = \sigma(20x_1 + 20x_2 - 30) \quad (\text{AND})$$

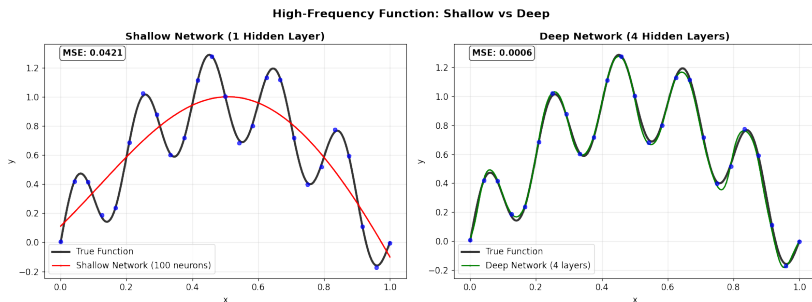
$$y = \sigma(20h_1 - 20h_2 - 10)$$

**Result:**  $x_1 \oplus x_2 = (x_1 \vee x_2) \wedge \neg(x_1 \wedge x_2)$ . Substitute the four inputs. Each  $\sigma(\pm 10)$  rounds to 1 or 0.

**General Principle:** Complex functions = composition of simple functions.

# Beyond XOR: High-Frequency Functions

**Test Case:**  $f(x) = \sin(\pi x) + 0.3 \sin(10\pi x)$



Shallow (100 neurons) vs Deep (4 layers, 25 each) networks

**Result:** The deep network reaches MSE 0.0006 against 0.0421 for the shallow one, using 2026 parameters against 301.

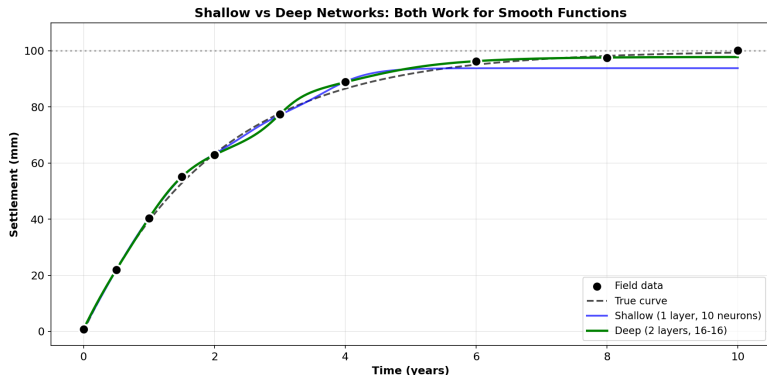
# Historical Timeline: From Crisis to Revolution

| Year        | Event                           | Impact                       |
|-------------|---------------------------------|------------------------------|
| 1943        | McCulloch-Pitts neuron          | Foundation laid              |
| 1957        | Rosenblatt's Perceptron         | First learning success       |
| <b>1969</b> | <b>Minsky &amp; Papert: XOR</b> | <b>Showed limits</b>         |
| 1970s-80s   | "AI Winter"                     | Funding dried up             |
| 1986        | Backpropagation                 | Enabled multi-layer training |
| 1989        | Universal Approximation         | Theoretical foundation       |
| 2006+       | Deep Learning Revolution        | Depth proves essential       |

**Lesson:** A perceptron with no hidden layer cannot represent XOR. One hidden layer can. Minsky and Papert showed the limit of the perceptron, and training multilayer networks stayed open until backpropagation.

# Shallow vs Deep on Consolidation

**Comparison:** 1 layer (10 neurons) vs 2 layers (16-16)



Both fit the data. Shallow uses 31 parameters, deep uses 321.

**When to go deep?** Complex boundaries, high-frequency, images.  
Otherwise 1-2 layers sufficient.

# Four Key Insights on Depth

## 1. Representation Efficiency

- For some function families a shallow network needs exponentially more neurons than a deep one (Telgarsky 2016, Eldan and Shamir 2016)
- For the smooth 1D targets in this deck the two are comparable

## 2. Feature Hierarchy

- Layer 1: Simple features (edges, patterns)
- Layer 2: Feature combinations (corners, textures)
- Layer 3+: Complex abstractions (objects, concepts)

## 3. Geometric Transformation

- Each layer performs coordinate transformation
- Deep networks "unfold" complex data manifolds

## 4. Compositional Learning

- Complex functions = composition of simple functions
- Build complexity incrementally

# Practical Recipe: 1D Consolidation

## Architecture:

- 2 hidden layers, 16-32 neurons each
- Activation: Tanh (bounded), ReLU (unbounded)

## Training:

- Optimizer: Adam, lr  $10^{-3}$  by default,  $10^{-2}$  for this 1D fit
- Epochs: 2000-3000
- Monitor validation loss for overfitting

## Results:

- Error against the true curve  $\approx 2$  to  $3 \text{ mm}^2$  with tanh
- Smooth predictions inside the training range
- Outside that range the fit is unreliable (03d)

# Summary and Conclusions

## Key Takeaways:

- Nonlinearity essential for complex patterns
- UAT: Single layer theoretically sufficient
- Practice: 1-2 layers for most problems
- Activation choice matters: Tanh for bounded, ReLU for unbounded
- Training: Adam + validation prevents overfitting
- XOR lesson: Depth creates feature hierarchies

## Applications:

- Consolidation settlement prediction
- Liquefaction classification
- Function approximation from sparse data
- Physics-informed neural networks (PINNs)

Thank you!

**Contact:**

Krishna Kumar

*krishnak@utexas.edu*

University of Texas at Austin